

NATIONAL ECONOMICS UNIVERSITY



Customer Churn Prediction and Campaign Analytics for Music Subscription Platforms

Course: Data-Driven Marketing

Group 6

Name	Student ID
Doan Tung Lam	11230555
Nguyen Manh Cuong	11230521
Tran Thu Hien	11230534
Do Ha Linh	11230557
Pham Minh Bao Ngoc	11230576

Class: DSEB 65B

Supervisor: Dr. Nguyen Tuan Long

Hanoi, April 2026

Abstract

This report implements a data-driven decision framework to optimize customer retention for the KKBox music streaming service. The pipeline covers raw data preprocessing, temporal feature engineering, model training with hyperparameter tuning, and a business-driven retention layer that converts predicted churn probabilities into per-customer voucher decisions. Feature engineering follows Bryan Gregory's first-place methodology, producing 53 behavioral and transactional signals. A LightGBM-XGBoost weighted ensemble achieves a validation AUC of 0.769. The retention framework identifies 154,415 high-priority customers for a March 2017 campaign with an estimated ROI of 33–108% depending on budget allocation. A/B testing design and dashboard prototypes are outlined as planned future work.

Contents

1	Introduction	4
1.1	Business Context	4
1.2	Objectives	4
1.3	Competition Benchmark	4
1.4	Report Structure	5
2	Data Understanding & Preprocessing	5
2.1	Data Sources	5
2.2	Preprocessing Steps	5
2.2.1	Transactions	5
2.2.2	Members	5
2.2.3	User Activity Logs	6
2.3	Data Quality Summary	6
3	Feature Engineering	6
3.1	Data Split and Churn Labels	6
3.1.1	Label Definition	7
3.2	Anti-Leakage Constraints	7
3.3	Temporal Feature Engineering Methods	7
3.4	Feature Groups	7
3.4.1	Member Features (10)	8
3.4.2	Transaction Features (30)	8
3.4.3	Log Features (13)	8
3.5	Feature Summary	8
4	Modeling	9
4.1	Setup	9
4.2	Logistic Regression Baseline	9
4.3	LightGBM	9
4.4	XGBoost	9
4.5	Ensemble and Calibration	10
4.6	Feature Importance	10
5	Retention Decision	10
5.1	Behavioral Weight Computation	10
5.2	Quantile-Based Risk Tiers	11
5.3	Customer Lifetime Value	12
5.4	Retention Cost and Break-Even	12
5.5	Threshold Sweep and Campaign Results	12
5.6	Strategic Segmentation	13

6	A/B testing	13
6.1	Why April Labels Are Unavailable	13
6.2	Part 1 — February 2017 Retrospective Backtest	13
6.2.1	Design	13
6.2.2	Backtest Results	14
6.3	Part 2 — Monte Carlo Simulation on March 2017	14
6.3.1	Simulation Rationale and Anchors	14
6.3.2	Two-Stage Bernoulli Process	15
6.3.3	Simulation Results	15
6.3.4	Economic Validation	16
7	Conclusion	16
7.1	Summary of Contributions	16
7.2	A/B Test Conclusion	17
7.3	Limitations	17

1 Introduction

1.1 Business Context

Customer churn—the voluntary cancellation of a subscription—is one of the most costly challenges for streaming platforms. KKBox, a music subscription service based in Taiwan, faces this challenge at scale: acquiring a new subscriber costs significantly more than retaining an existing one, and each churned user represents a lost revenue stream proportional to their remaining expected lifetime.

This project uses data from the WSDM Cup 2018 Kaggle competition, which provides historical transaction records, member demographics, and listening activity logs from KKBox. The task is to predict, at the end of each month, which users will not renew their membership within 30 days of expiry.

1.2 Objectives

The project pursues two parallel goals:

- **Predictive:** Build a model that accurately ranks users by churn probability, evaluated by AUC on the February 2017 validation set and benchmarked against the competition leaderboard.
- **Prescriptive:** Convert churn probabilities into actionable retention decisions—which customers to contact, with what voucher, and in what priority order—grounded in Customer Lifetime Value and individual break-even economics.

1.3 Competition Benchmark

Bryan Gregory's Winning Approach

Bryan Gregory won the WSDM Cup 2018 with a final LogLoss of 0.07974. His key contributions are: (1) the *relative refactoring method* for temporal features—converting dates to days elapsed since the prediction period start so features are comparable across train, validation, and test; (2) iterative cross-validation feature selection from 208 candidates down to 76; and (3) a weighted ensemble of XGBoost (88%) and LightGBM (12%). Over 80% of his features are derived from user activity logs and transaction records.

Our implementation replicates his feature engineering approach on the public dataset and achieves a validation AUC of 0.769. The primary remaining gap is caused by a column name discrepancy in the log data (`num_uniq` vs `num_uniq_songs`), which results in all song-based features being zero in the current run.

1.4 Report Structure

The report proceeds as follows. Section 2 covers data understanding and preprocessing. Section 3 describes temporal feature engineering and the label construction methodology. Section 4 presents the modeling approach and results. Section 5 details the retention decision framework and campaign design. Section 6 outlines the A/B testing plan. Section 7 concludes with findings and future work.

2 Data Understanding & Preprocessing

2.1 Data Sources

The dataset consists of five raw files covering transactions, member profiles, and listening activity. Table 1 summarizes each file.

Table 1: Raw Data Files

File	Rows	Description
transactions.parquet	547,746	Subscriptions, Jan 2015–Jan 2017
transactions_v2.csv	1,431,009	Additional transactions, Jan–Mar 2017
members_v3.csv	6,769,473	User demographics and registration info
user_logs.parquet	106,543	Historical daily listening (22,443 users)
user_logs_v2.parquet	396,362	March 2017 listening (316,345 users)

Key characteristic: Dates are stored as YYYYMMDD integers (e.g., 20170131 represents January 31, 2017) and must be parsed before use. The two transaction files cover overlapping periods and are merged, with duplicates removed.

2.2 Preprocessing Steps

2.2.1 Transactions

The two transaction files are concatenated into a single table of 1,978,751 rows. The following cleaning rules are applied:

- Rows with null `transaction_date` or `membership_expire_date` are dropped.
- For cancellation rows (`is_cancel = 1`, `payment_plan_days = 0`), `actual_amount_paid` is set to zero, since cancelled transactions carry no payment.
- An expiry gap (`expire_gap_days`) is computed as the difference between expiry date and transaction date. Values exceeding 365 days are flagged as extreme (`is_extreme_expiry = 1`).

2.2.2 Members

The member table contains demographic information that requires cleaning:

- **bd** (birth year / age): values outside $[10, 80]$ are replaced with the median, and a binary **bd_missing** flag is created.
- **city, gender, registered_via**: missing values are filled with the string “Unknown” and corresponding missing flags (**city_missing**, etc.) are added.
- **registration_init_time**: decomposed into **registration_year, registration_month, registration_day** and then dropped.

2.2.3 User Activity Logs

The two log files are kept as separate branches rather than merged. The overlap between them is only 4,081 users (18% of the historical log, 1.3% of the March log), confirming that they represent different user cohorts rather than a continuous history.

Log Coverage Limitation

Only 1.1% of the January 2017 training population (10,708 / 992,931 users) appears in the historical log file. This is a known limitation of the public dataset release — KKBox’s internal data includes logs for all users. Zero values in log features are retained as valid signals: a user with no listening history is more likely to churn than an active listener.

2.3 Data Quality Summary

After preprocessing, the clean data contains:

- **Transactions**: 1,978,751 rows, 1,315,711 unique users, no null dates.
- **Members**: 6,769,473 records, 12 columns, no null values in core fields.
- **Logs (historical)**: 106,543 rows, 22,443 users, date range Jan 2015 – Feb 2017.
- **Logs (March)**: 396,362 rows, 316,345 users, date range Mar 2017 only.

3 Feature Engineering

3.1 Data Split and Churn Labels

The time-based split follows Bryan Gregory’s exact configuration to ensure leakage-free evaluation. Table 2 summarizes the three splits.

Table 2: Train / Validation / Inference Split

Split	Month	Users	Churn rate	Label source
Train	Jan 2017	992,931	6.39%	<code>train.csv</code> (official)
Validation	Feb 2017	970,960	8.99%	<code>train_v2.csv</code> (official)
Inference	Mar 2017	907,471	—	<code>sample_submission_v2.csv</code>

Churn labels are taken from the competition’s official files rather than reconstructed internally. This guarantees alignment with competition evaluation.

3.1.1 Label Definition

A user is labelled `is_churn = 1` if they do not renew within 30 days of the `membership_expire_date` of their most recent effective subscription. “Effective” means the state reconstructed at the cutoff using the Scala-aligned sort:

```
sort: transaction_date ASC, is_cancel ASC, membership_expire_date ASC  
  
groupby(msno).last() ⇒ effective state per user at cutoff
```

This logic replicates the official `WSDMChurnLabeller.scala`: the latest transaction wins; on the same date, renewals are preferred over cancellations; among the same type, the longer expiry is taken.

3.2 Anti-Leakage Constraints

All features are computed strictly on data with `transaction_date ≤ cutoff_date`. The cutoff is the last day of the prediction month (e.g., 2017-01-31 for training). The renewal search for label construction is the sole exception: finding a transaction after `last_expire` is part of label logic, not feature logic.

3.3 Temporal Feature Engineering Methods

Bryan Gregory [1] identifies two complementary approaches:

Relative Refactoring Method

A date field is converted to the number of days elapsed since the start of the prediction period. This makes features comparable across train, validation, and test despite covering different calendar months.

Example: `days_since_last_txn = cutoff_date - last_transaction_date`

Absolute Method

The raw date is retained as a YYYYMMDD integer. Used when the calendar date carries signal that elapsed time does not—for example, if users who registered after a holiday promotion have distinct churn behavior.

Example: `registration_date_abs = 20170101`

3.4 Feature Groups

A total of **53 features** are engineered across three source groups.

3.4.1 Member Features (10)

Demographic features (`bd`, `city`, `gender`, `registered_via`), binary missing flags for each, `days_since_reg` (relative method), and `registration_date_abs` (absolute method).

3.4.2 Transaction Features (30)

Key features and their reference to Bryan Gregory’s published top-10 are shown in Table 3.

Table 3: Key Transaction Features

Feature	Bryan ref.	Description
<code>last_plan_days</code>	#1 (6.61%)	Most recent subscription plan length
<code>cancel_to_plandays_ratio</code>	#6 (3.81%)	Cancel count / total plan days
<code>active_no_cancel_flag</code>	#9 (2.80%)	>1 transaction last month, no cancel
<code>last_is_cancel</code>	Paper	Last transaction was a cancellation
<code>cancel_in_last_month</code>	Paper	Any cancellation in last 30 days
<code>avg_cancel_per_month</code>	Paper	Historical cancellations per month
<code>days_since_last_cancel</code>	Paper	Days since most recent cancellation
<code>recency_to_plan_ratio</code>	Higher-order	<code>days_since_last_txn</code> / <code>avg_plan_days</code>
<code>spend_rate_per_day</code>	Higher-order	<code>total_amount_paid</code> / <code>tenure_days</code>

3.4.3 Log Features (13)

User activity features across multiple time windows:

- All-history: `total_secs_played`, `total_uniq_songs`, `num_logins`
- 14-day window (Bryan #2): `secs_played_14d`, `logins_14d`
- 30-day window: `secs_played_30d`, `logins_30d`, `logins_90d`
- Prior-prior month (Bryan #5): `secs_played_60_30d`
- Trend deltas (Bryan #2, #4): `delta_secs_30d_vs_prior`, `delta_songs_14d_vs_30d`
- Recency: `days_since_last_login`, `days_since_significant_usage`
- Binary engagement flag: `has_log_history`

3.5 Feature Summary

Table 4: Feature Engineering Summary

Source	Features	Bryan coverage
Member features	10	Demographic + registration date
Transaction features	30	Covers 7 of Bryan’s Top 10
Log features	13	Covers 5 of Bryan’s Top 10
Total	53	vs 76 in Bryan’s final model

The output `master_model_table.parquet` contains 1,963,891 rows (train + validation) with 53 feature columns and 6 meta columns. Zero NaN values and no meta columns in the feature matrix.

4 Modeling

4.1 Setup

All models use the same 54-feature matrix (53 original plus `has_ever_cancelled` added after the `days_since_last_cancel` sentinel fix). Categorical columns are ordinal-encoded; numeric columns are median-imputed using training-set statistics only. Class imbalance (14.6:1) is handled via `is_unbalance=True` for LightGBM and `scale_pos_weight=14.64` for XGBoost.

4.2 Logistic Regression Baseline

Logistic Regression with `class_weight='balanced'`, L2 regularisation, and the SAGA solver confirms that the feature set carries linear signal (AUC 0.702). This baseline rules out trivially bad features before investing in gradient boosting.

4.3 LightGBM

LightGBM uses leaf-wise tree growth, which achieves greater depth per tree than level-wise approaches and is well-suited to categorical and sparse features.

Critical Fix: num_uniq Column Name

Feature engineering referenced the unique-songs column as `num_uniq_songs`; the actual column name is `num_uniq`. Python returned zeros silently. Five song-based features were identically zero for all users, causing LGBM to exhaust available signal in 6 rounds instead of 214. After fixing the column name: best_iteration rose from 6 to 214; the inference P_churn range widened from [0.065, 0.134] to [0.071, 0.181]; and campaign ROI improved from 33.0% to 44.7%.

Hyperparameter optimisation uses Optuna TPE sampler, 100 trials, objective = maximise validation AUC. The best trial was found at trial 98, confirming that 50 trials would have been insufficient.

4.4 XGBoost

XGBoost uses level-wise growth with `tree_method='hist'` (Bryan Gregory's explicit recommendation for speed at high dimensionality). XGBoost achieves higher AUC (0.766) but worse LogLoss (0.551) than LightGBM (AUC 0.766, LogLoss 0.290). Since LogLoss governs probability calibration and all downstream CLV calculations depend on accurate probabilities, LightGBM is the primary model.

4.5 Ensemble and Calibration

$$\hat{p}_{\text{churn}} = w_{\text{LGBM}} \cdot \hat{p}_{\text{LGBM}} + w_{\text{XGB}} \cdot \hat{p}_{\text{XGB}} \quad (1)$$

A grid search over $w_{\text{LGBM}} \in [0, 1]$ in 5% steps yields the optimal blend of 95% LightGBM + 5% XGBoost (AUC 0.769). Bryan Gregory used 12/88 because his XGBoost dominated; our LightGBM has better calibration.

Raw ensemble probabilities are calibrated using isotonic regression fitted on the validation set. The calibrated mean prediction (8.99%) exactly matches the observed February churn rate. The 0.75 multiplier used by competition top solutions is deliberately excluded: it requires knowing the test-set churn rate in advance, which is unavailable in a real deployment context.

Table 5: Model Performance on Validation Set (February 2017)

Model	Val AUC	Val LogLoss	Notes
Logistic Regression	0.702	0.606	Linear baseline
LightGBM	0.766	0.290	Best LogLoss; best_iter 214
XGBoost	0.766	0.551	Good ranking
Ensemble (95/5)	0.769	—	Grid-searched blend
After isotonic calib.	0.769	0.257	Mean pred = 8.99%
<i>Bryan Gregory (1st place): LogLoss 0.07974</i>			

4.6 Feature Importance

The top features by LGBM gain are `registered_via` (42.0%), `registration_date_abs` (12.6%), `total_amount_paid` (9.9%), `days_since_reg` (7.3%), and `last_is_auto_renew` (7.0%). The high concentration on `registered_via` reflects residual log-feature sparsity; song-based features are expected to gain importance upon full log coverage.

5 Retention Decision

The retention layer converts continuous churn probabilities into per-customer economic decisions. The core principle: a voucher should be sent only when the expected economic benefit of retention exceeds the voucher cost.

5.1 Behavioral Weight Computation

Rather than assuming fixed multipliers, save-rate adjustments are derived from actual churn-rate differences in training data. For each binary flag i :

$$\text{relative_effect}_i = \frac{\text{churn_rate}(i=1) - \text{churn_rate}(i=0)}{\text{overall_churn_rate}} \quad (2)$$

$$w_i = \text{clip}(-\text{relative_effect}_i \times 0.25, -0.25, +0.25) \quad (3)$$

The sign convention: a behavior that *reduces* churn yields a positive weight (increases save rate); a behavior that *increases* churn yields a negative weight.

Table 6: Data-Driven Behavioral Weights (computed from training data)

Behavior	Feature	Churn (=0)	Churn (=1)	Weight
Auto-renew active	<code>last_is_auto_renew</code>	8.9%	5.1%	+0.125
Recent cancellation	<code>last_is_cancel</code>	7.6%	14.6%	-0.225
Has log history	<code>has_log_history</code>	7.7%	8.3%	-0.021
Active, no cancel	<code>active_no_cancel</code>	7.5%	62.1%	-0.250
High listening vol.	<code>total_secs > Q75</code>	7.7%	8.3%	-0.021
Long plan user	<code>avg_plan_days ≥ 90</code>	7.4%	72.4%	-0.250

Counterintuitive Findings

Long-plan users churn at 72.4% — $8.5\times$ the population average. These are promotional buyers who purchased a 90-day discount and do not renew at the standard price. The commitment signal is an adverse selection signal.

Active-no-cancel users churn at 62.1%. They are passive churners approaching natural membership lapse without explicitly cancelling — looking healthy by surface metrics while silently departing.

Both patterns are only discoverable through data-driven weight computation. Assumed multipliers would have marked both groups as safe.

5.2 Quantile-Based Risk Tiers

Fixed thresholds (e.g. 0.30, 0.50, 0.70) become meaningless when the model produces a narrow or bimodal output. Quantile-based boundaries adapt to the actual distribution:

Table 7: Risk Tier Configuration ($Q_1 = 0.026$, $Q_2 = 0.040$, $Q_3 = 0.060$)

Tier	P_churn	Users	Voucher	Base save rate
Stable	$< Q_1$	173,442 (19.1%)	0%	0%
Medium	$[Q_1, Q_2)$	210,125 (23.2%)	5%	8%
High	$[Q_2, Q_3)$	119,384 (13.2%)	10%	15%
Critical	$\geq Q_3$	404,520 (44.6%)	20%	7%

5.3 Customer Lifetime Value

$$\text{monthly_revenue}_i = \frac{\text{avg_amount_paid}_i}{\text{avg_plan_days}_i/30} \quad (4)$$

$$\text{expected_lifetime} = \frac{1}{\text{val_churn_rate}} = \frac{1}{0.0899} \approx 11.1 \text{ months} \quad (5)$$

$$\text{CLV}_i = \text{monthly_revenue}_i \times \text{expected_lifetime} \quad (6)$$

The validation churn rate is read from the training data, not hard-coded, so CLV updates automatically upon model retraining. Median CLV is \$1,434; maximum is \$21,881.

5.4 Retention Cost and Break-Even

$$\text{retention_cost}_i = \min(\text{avg_amount_paid}_i \times \text{voucher_pct}, \$200) \quad (7)$$

A voucher is sent only when both conditions hold simultaneously:

1. Risk tier is not Stable.
2. $\mathbb{E}[\text{profit}_i] = p_{\text{churn},i} \cdot \text{CLV}_i \cdot \text{save_rate}_i - \text{retention_cost}_i > 0$

The per-customer break-even probability, derived from condition 2, is:

$$p_i^* = \frac{\text{retention_cost}_i}{\text{CLV}_i \times \text{save_rate}_i} \quad (8)$$

5.5 Threshold Sweep and Campaign Results

Table 8: Threshold Sweep — Selected Budget Scenarios

Threshold	Users	Cost	Net profit	ROI
0.054	78,540	\$2,449,055	\$1,105,301	45.1%
0.177	73,527	\$2,376,898	\$1,099,599	46.3%
0.300	61,124	\$2,122,668	\$995,772	46.9%
0.456	5,935	\$251,190	\$120,650	48.0%
Selected	84,420	\$2,517,531	\$1,126,258	44.7%

Before vs After Model Fix

Metric	Before fix	After fix
LGBM best_iteration	6	214
P_churn max (inference)	0.134	0.181
Retention targets	154,415	84,420 (−45%)
Campaign cost	\$2,891,910	\$2,517,531
Net benefit	\$954,847	\$1,126,258 (+18%)
Campaign ROI	33.0%	44.7%

Better model quality means fewer but more precise targets, lower cost, and higher net benefit.

5.6 Strategic Segmentation

Table 9: Strategic Segments and Retention Actions

Segment	Users	Net benefit	ROI	Action
Critical (profitable)	75,787	\$1,105,002	45.7%	Max 20% voucher; premium rescue
High (profitable)	6,199	\$19,989	25.2%	Standard 10% voucher
Medium (profitable)	2,434	\$1,267	7.2%	Light 5% voucher
Stable	173,442	\$0	—	Monitor only; no spend

Budget recommendation: Running Critical-only captures 98% of the net benefit at 54% of full-reach cost, with a 45.7% ROI.

6 A/B testing

6.1 Why April Labels Are Unavailable

The March 2017 inference population has membership expiry dates in April 2017. Observing renewal decisions requires April/May 2017 transaction data. Confirmed: `transactions_v2.csv` contains zero rows with `transaction_date` $\geq$ 2017-04-01, despite 1,025,980 memberships expiring in April. A/B testing is therefore conducted in two parts: Part 1 uses February 2017 actual labels; Part 2 uses a principled Monte Carlo simulation anchored to observable signals.

6.2 Part 1 — February 2017 Retrospective Backtest

6.2.1 Design

The February 2017 population (970,960 users) has official churn labels in `train_v2.csv`. Non-Stable users (817,388) are assigned to three arms using deterministic MD5 hashing of `campaign_id:msno`, ensuring reproducibility across runs.

Table 10: Experimental Arms (non-Stable February 2017 users)

Arm	Share	Users	Treatment
Control (C)	20%	163,304	No voucher
Treatment A (T_A)	40%	326,896	Tier-based voucher (5/10/20%)
Treatment B (T_B)	40%	327,188	Flat 10% voucher

6.2.2 Backtest Results

Table 11: Retrospective A/B Backtest Results (February 2017)

Arm	Renewal rate	Lift vs C	p-value	Significant?
Control (C)	89.34%	—	—	—
T_A (tier)	89.33%	−0.01%	0.560	No
T_B (flat)	89.29%	−0.05%	0.706	No

Table 12: Save Rate Validation by Tier (Control arm, actual labels)

Tier	Control renewal	Assumed save rate	Target renewal	Achievable?
Medium	99.43%	8.0%	107.4%	No
High	99.51%	15.0%	114.5%	No
Critical	96.68%	7.0%	103.7%	No

Interpreting the Null Result

The null result is informative, not a failure. Medium and High tier users renew at 99.4–99.5% in the control arm. No voucher can improve on 99.4% because the ceiling is 100%. The model assigns users with $P_{\text{churn}} = 0.03\text{--}0.06$ to these tiers, but their actual churn is only 0.5%. Model discrimination is the binding constraint, not voucher design.

Furthermore, High tier churns at 0.49% — *lower* than Medium at 0.57%. This tier inversion confirms the model cannot meaningfully rank users within the 0.038–0.074 P_{churn} range.

6.3 Part 2 — Monte Carlo Simulation on March 2017

6.3.1 Simulation Rationale and Anchors

The simulation answers: *if the model correctly identifies churners and vouchers have a realistic save rate, what would the A/B result look like?* Every parameter is anchored to a real observable:

- **Target churn 6.7%:** top competition solutions improved scores by multiplying predictions by 0.75, implying test churn $\approx 8.99\% \times 0.75 \approx 6.7\%$.

- **Tier-conditional churn:** from February actual labels — Critical 5.11%, Medium 0.57%, High 0.49%.
- **Treatment effect:** Bernoulli draw with Critical save rate 15%, Medium/High 3%.

6.3.2 Two-Stage Bernoulli Process

$$\tilde{p}_i = p_{\text{churn},i} \times \frac{\text{target_churn}}{\bar{p}_{\text{churn}}} \quad (9)$$

$$\text{is_churn_base}_i \sim \text{Bernoulli}(\tilde{p}_i) \quad (10)$$

$$\text{is_saved}_i \sim \text{Bernoulli}(\text{save_rate}_{\text{tier}}) \quad \text{for churners in T_A / T_B} \quad (11)$$

6.3.3 Simulation Results

Table 13: Monte Carlo Simulation Results Across 5 Seeds

Seed	Pop. churn	T_A lift	p-value	Significant?
42	6.67%	+0.96%	0.00000	Yes
123	6.70%	+1.09%	0.00000	Yes
999	6.70%	+1.09%	0.00000	Yes
2017	6.72%	+1.09%	0.00000	Yes
7	6.70%	+1.00%	0.00000	Yes
Average	6.70%	+1.05%	0.00000	All significant

Table 14: Sensitivity Analysis — Minimum Viable Critical Save Rate

Critical save rate	T_A lift	p-value	Significant?
5%	+0.23%	0.00403	Yes
10%	+0.60%	0.00000	Yes
15%	+0.96%	0.00000	Yes
20%	+1.34%	0.00000	Yes
30%	+1.96%	0.00000	Yes

Significant lift is achieved at every tested save rate from 5% upward. The minimum viable Critical save rate is **5%** — a very conservative assumption in the retention literature.

6.3.4 Economic Validation

Table 15: Backtest vs Simulation Economic Comparison

Scenario	Backtest (real)	Simulation
Population	Feb 2017 non-Stable	Mar 2017 (907,471)
T_A renewal	89.33%	93.0%
Control renewal	89.34%	92.0%
Incremental lift	−0.01%	+1.05%
Campaign cost (T_A)	\$1,656,503	\$1,656,503
Net benefit	−\$1,656,503	+\$4,463,497
ROI	−100%	+269.5%
Verdict	Do not ship	Would ship

The retention framework is economically sound. Model discrimination quality is the sole gap between the null backtest and a profitable live campaign.

7 Conclusion

7.1 Summary of Contributions

This project delivers a complete churn prediction and retention optimization pipeline for KKBox, from raw data cleaning through A/B experiment design. Four key contributions stand out:

1. **Temporal feature engineering:** 54 features built using Bryan Gregory’s relative refactoring and absolute temporal methods. The pipeline is fully leakage-proof with all features computed at the prediction cutoff.
2. **Ensemble model:** A LightGBM–XGBoost weighted blend (95/5) achieves validation AUC 0.769 and calibrated LogLoss 0.257 after correcting a single column-name error that previously reduced LGBM convergence from 214 iterations to 6.
3. **Data-driven retention economics:** Individual break-even thresholds replace a universal rule. Each voucher decision is grounded in the user’s own CLV, behavioral save-rate adjustment, and retention cost. The result is a campaign targeting 84,420 users with ROI 44.7% — up from 33.0% before the model fix.
4. **Behavioral insights:** Long-plan users churn at 72.4% and active-no-cancel users at 62.1% — both counterintuitive findings only discoverable through data-driven weight computation. These have direct product implications: long-plan promotions attract low-loyalty buyers, and “active” users may be passive churners silently approaching expiry.

7.2 A/B Test Conclusion

The retrospective backtest on February 2017 produces a null result (T_A lift = -0.01% , $p = 0.560$). This is the correct outcome: Medium and High tier users renew at 99.4–99.5% naturally, making the assumed save rates physically unachievable. The result is a model diagnostic, not a campaign failure.

The Monte Carlo simulation on March 2017, anchored to leaderboard evidence (implied churn $\approx 6.7\%$), shows significant lift across all five seeds (average $+1.05\%$, significant at Critical save rates as low as 5%). The retention framework is economically valid; model discrimination quality is the binding constraint.

7.3 Limitations

- **Log coverage:** 98.9% of training users have zero log features in the public dataset. Song-based features improve discrimination significantly but are unavailable at scale from the public release.
- **April labels:** March 2017 outcomes are unobservable from the public dataset. A live A/B test requires April 2017 transaction data not released by Kaggle.
- **CLV assumption:** Expected lifetime uses the February churn rate (8.99%) as a single-period estimate. A survival analysis approach would provide a more robust multi-period CLV.

References

- [1] B. Gregory, “Predicting Customer Churn: Extreme Gradient Boosting with Temporal Data,” *arXiv:1802.03396*, 2018.
- [2] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proc. 22nd ACM SIGKDD*, pp. 785–794, 2016.
- [3] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [4] T. Akiba et al., “Optuna: A Next-generation Hyperparameter Optimization Framework,” *Proc. 25th ACM SIGKDD*, pp. 2623–2631, 2019.
- [5] KKBox and Kaggle, “WSDM – KKBox’s Churn Prediction Challenge,” *Kaggle Competition*, 2018.